

Genetic Algorithm for Carbon-Aware Hyperparameter Optimization of Neural Networks

Preni Avanesi^{*†}, Lorenzo Carnevale^{*‡}

^{*}University of Messina, Messina, Italy

[†]preni.avanesi@studenti.unime.it

[‡]lcarnevale@unime.it

Abstract

...

Index Terms

Hyperparameter Optimization, Genetic Algorithm, NSGA-II, Carbon Emissions, Multi-Layer Perceptron, CodeCarbon, Pareto Front, Bio-Inspired Computing

I. INTRODUCTION

THE rapid growth of deep learning has enabled significant advances across a wide range of applications, including image recognition, natural language processing, healthcare, and finance. However, the performance of neural networks is highly sensitive to the choice of hyperparameters—configuration settings such as learning rate, batch size, and network architecture that are specified before training and are not directly learned from data. Finding an effective combination of hyperparameters, a process known as Hyperparameter Optimization (HPO), remains a fundamental challenge in machine learning practice [1].

A. Problem Identification

Traditional HPO methods such as Grid Search and Random Search suffer from important limitations. Grid Search exhaustively evaluates all possible hyperparameter combinations, causing computational cost and energy consumption to grow rapidly as the search space expands. Random Search is generally more efficient, but it lacks a systematic mechanism to guide exploration toward promising regions of the search space. Both methods primarily focus on predictive performance and typically ignore the environmental cost of repeated model training.

The environmental impact of machine learning has become an increasingly important research topic. Studies have shown that training large-scale machine learning models can generate substantial CO₂ emissions [2]. Although the neural networks considered in this work are considerably smaller than modern foundation models, repeated hyperparameter evaluations still consume computational resources and energy. Consequently, there is growing interest in optimization approaches that simultaneously consider model accuracy, computational efficiency, and environmental sustainability.

B. Research Objective

The objective of this work is to investigate whether multi-objective evolutionary optimization can identify hyperparameter configurations that simultaneously reduce classification error, training time, and CO₂ emissions while maintaining competitive predictive performance. Specifically, the study evaluates whether the Non-dominated Sorting Genetic Algorithm II (NSGA-II) can provide an effective and environmentally conscious alternative to traditional HPO methods such as Grid Search and Random Search.

C. Contributions

This project makes the following contributions:

- Design and implementation of a multi-objective hyperparameter optimization framework based on NSGA-II for carbon-aware optimization of Multi-Layer Perceptron (MLP) classifiers on the Wine Quality dataset [3].

- Integration of the CodeCarbon library (`OfflineEmissionsTracker`) for consistent and reproducible CO₂ emission estimation during model training using Italy’s fixed carbon intensity of 0.233 kg CO₂/kWh.
- Development of a rigorous experimental protocol based on stratified train/validation/test splitting to prevent data leakage during hyperparameter search and ensure unbiased model evaluation.
- Implementation of an evaluation cache that reuses previously computed objective values for duplicate hyperparameter configurations, substantially reducing the number of expensive model trainings during evolutionary search.
- Comparative evaluation against Grid Search and Random Search baselines using multiple criteria, including hypervolume, test accuracy, runtime, CO₂ emissions, number of evaluations, and Pareto-front size.
- Application of a weighted fitness function with equal objective weights ($w_1 = w_2 = w_3 = \frac{1}{3}$) to select a final configuration from the Pareto front and compare balanced trade-off solutions across optimization methods.
- Visualization and analysis of Pareto fronts through two-dimensional objective-space projections, illustrating the trade-offs between predictive performance, training time, and carbon emissions across all optimization methods.

II. RELATED WORKS

Hyperparameter optimization has been an active research area in machine learning for several decades. This section reviews the most relevant approaches and positions our work within the existing literature.

A. Traditional HPO Methods

Grid Search is the most straightforward HPO method, performing an exhaustive search over a predefined hyperparameter grid. While guaranteed to find the globally optimal configuration within the grid, it scales poorly with the number of hyperparameters due to the combinatorial explosion of the search space — evaluating all n^k combinations for k hyperparameters each with n values. Random Search [4] addresses this limitation by sampling configurations uniformly at random, demonstrating that it can match or outperform Grid Search with significantly fewer evaluations, particularly when only a subset of hyperparameters significantly affects model performance. Neither method, however, provides any mechanism to explicitly optimize for computational cost or environmental impact.

B. Bayesian Optimization

Bayesian Optimization [5] represents a more sophisticated approach, building a probabilistic surrogate model of the objective function and using an acquisition function to select the most promising configurations to evaluate next. Methods such as the Tree-structured Parzen Estimator (TPE) and Sequential Model-Based Algorithm Configuration (SMAC) have shown strong performance on benchmark HPO tasks. More recently, multi-objective extensions such as MOTPE [6] have been proposed to handle competing objectives simultaneously. However, none of these methods explicitly account for carbon emissions during the search process, treating environmental cost at most as a post-hoc measurement.

C. Evolutionary Approaches

Evolutionary algorithms have been applied to HPO since the early days of neural architecture search. Genetic Algorithms [7] simulate the process of natural selection to evolve populations of candidate solutions toward better performance. More recently, multi-objective evolutionary algorithms such as NSGA-II [8] have been applied to neural architecture search — for instance, NSGA-Net [9] demonstrated simultaneous optimization of accuracy and computational complexity in image classification tasks. NSGA-II’s non-dominated sorting mechanism produces a Pareto front of trade-off solutions rather than a single optimum, making it naturally suited for problems with competing objectives. However, existing applications of NSGA-II to HPO do not incorporate carbon awareness as an explicit optimization objective.

D. Carbon-Aware Machine Learning

The environmental impact of machine learning has gained significant attention in recent years. Strubell2019 demonstrated that training large NLP models can emit as much CO₂ as five cars over their lifetime, motivating the development of energy-efficient training strategies. Schwartz2020 further argued that computational cost should be reported alongside predictive performance as a standard practice in machine learning research — a principle this work adopts. Tools such as CodeCarbon [10] have been

developed to measure and report the carbon footprint of model training in real time. However, most existing HPO frameworks treat carbon emissions as a post-hoc measurement rather than an optimization objective. Our work addresses this gap by integrating carbon tracking directly into the multi-objective fitness function of NSGA-II, enabling carbon-aware hyperparameter selection during the search process itself.

Despite the success of existing HPO methods, three limitations remain evident. First, traditional methods primarily focus on predictive performance and ignore environmental cost. Second, most carbon-tracking tools are used only for post-hoc reporting rather than optimization. Third, few studies explicitly treat validation error, training time, and CO₂ emissions as simultaneous optimization objectives. These observations motivate the framework proposed in this work.

E. Comparison with Related Works

Table I summarizes the key features of related HPO methods compared to our proposed approach. Our method is the only one that simultaneously addresses all three dimensions: multi-objective optimization, carbon awareness, and Pareto front generation.

Table I: Comparison with the state of the art.

Method	Multi-Objective	Carbon-Aware	Pareto Front
Grid Search	No	No	No
Random Search [4]	No	No	No
Bayesian Opt. [5]	No	No	No
MOTPE [6]	Yes	No	Yes
NSGA-II [8]	Yes	No	Yes
Our Method	Yes	Yes	Yes

III. MATERIAL AND METHODS

In the following, we discuss the materials and methodologies used to achieve the goals described above.

A. Material

Dataset. Experiments are conducted on the Wine Quality dataset from the UCI Machine Learning Repository [3]. The dataset contains 6,497 wine samples described by 11 physicochemical features: fixed acidity, volatile acidity, citric acid, residual sugar, chlorides, free sulfur dioxide, total sulfur dioxide, density, pH, sulphates, and alcohol content. The target variable is a quality score ranging from 3 to 9, assigned by human wine experts. For this work, the quality score is binarized: wines with quality ≥ 6 are labeled as *good* (class 1), while those with quality < 6 are labeled as *bad* (class 0).

The dataset is partitioned into three disjoint sets using stratified sampling to preserve class distribution:

- **Training set:** 70% — 4,550 samples, used to fit each MLP configuration
- **Validation set:** 15% — 972 samples, used to evaluate fitness during search
- **Test set:** 15% — 975 samples, reserved exclusively for final evaluation

The test set is used only once after the best configuration is selected, preventing test set leakage during the search process. Features are standardized using Z-score normalization:

$$x' = \frac{x - \mu}{\sigma} \quad (1)$$

where μ and σ are the mean and standard deviation computed from the training set only, applied identically to all three sets to prevent data leakage.

After the best hyperparameter configuration is selected via MCDM, the final model is retrained on the combined training and validation sets (5,522 samples). Since the validation set is no longer required for fitness evaluation at this stage, it is added to the training data, providing approximately 21% more samples ($972/4,550 \approx 21\%$) for the final model and improving generalization on the test set.

Model Architecture. The classifier used in this work is a Multi-Layer Perceptron (MLP), implemented using `MLPClassifier` from scikit-learn . The MLP consists of three layer types:

- **Input Layer:** 11 neurons, one per physicochemical feature. No activation is applied.
- **Hidden Layers:** Between 1 and 6 layers (defined by the chromosome), each with 32–512 neurons. To keep the search space tractable, all hidden layers within a configuration are assigned the same number of neurons, reducing the architecture search space to 30 distinct configurations. The ReLU activation function is applied:

$$\text{ReLU}(x) = \max(0, x) \quad (2)$$

ReLU is chosen for its computational efficiency, resistance to the vanishing gradient problem, and widespread adoption in MLP architectures.

- **Output Layer:** 1 neuron with Logistic (Sigmoid) activation:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

The sigmoid output is interpreted as the probability of the wine being *good*. A threshold of 0.5 is applied for final classification.

Loss Function. The MLP minimizes the Binary Cross Entropy loss:

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] \quad (4)$$

where $y_i \in \{0, 1\}$ is the true label, $\hat{y}_i$ is the predicted probability, and N is the number of samples. Cross Entropy is preferred over Mean Squared Error for classification as it penalizes confident wrong predictions more heavily, yielding faster and more stable convergence.

Optimizer. The MLP is optimized using **Adam** (Adaptive Moment Estimation), the default solver in scikit-learn's `MLPClassifier`. Adam maintains per-parameter adaptive learning rates using first and second moment estimates:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (5)$$

$$\theta \leftarrow \theta - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \quad (6)$$

where $\beta_1 = 0.9$, $\beta_2 = 0.999$, and η is the learning rate treated as a hyperparameter optimized by the GA. Adam is preferred over SGD for its faster convergence and robustness to hyperparameter choices.

Training Settings. Two training configurations are used throughout this work:

- **During hyperparameter search:** `max_iter=30`, `early_stopping=False`, `solver='adam'`. A low epoch count keeps each evaluation fast, reducing total search time and CO₂ emissions. Early stopping is disabled to ensure all evaluations train on the complete training set consistently.
- **Final model retraining:** `max_iter=100`. After the best configuration is selected via MCDM, the model is retrained on the combined training and validation sets (5,522 samples) with more epochs to maximize test accuracy.

Carbon Emissions Measurement. Carbon emissions are measured using the CodeCarbon library [10]. The CO₂ footprint is estimated as:

$$\text{CO}_2 \text{ (kg)} = P \text{ (kW)} \times t \text{ (h)} \times I \text{ (kg CO}_2\text{/kWh)} \quad (7)$$

where P is CPU power consumption, t is execution time, and I is the carbon intensity of the local energy grid. Each component is obtained as follows:

CPU Power (P): estimated from CPU utilization and Thermal Design Power (TDP):

$$P = \text{TDP} \times \text{utilization} \approx 80 \text{ W} \times 0.5 = 0.04 \text{ kW} \quad (8)$$

Execution Time (t): wall-clock time wrapping each MLP training call:

$$t \text{ (h)} = \frac{\text{training time (s)}}{3600} \quad (9)$$

Carbon Intensity (I): OfflineEmissionsTracker with `country_iso_code="ITA"` applies Italy's fixed carbon intensity of 0.233 kg CO₂/kWh. Offline mode eliminates the 1–2 second network overhead per call that would add over 50 minutes of latency across 1,494 NSGA-II evaluations, and ensures consistent reproducible measurements. Table II reports carbon intensities for reference.

Table II: Carbon intensity by country (kg CO₂/kWh).

Country	Intensity	Main energy source
Norway	0.013	95% hydropower
France	0.056	70% nuclear
Italy	0.233	Mix renewable + gas
Germany	0.338	Mix coal + solar
USA	0.386	Mix coal + gas + renewable
Poland	0.773	70% coal

As a concrete example, a training lasting 1.79 seconds produces:

$$\text{CO}_2 = 0.04 \text{ kW} \times \frac{1.79}{3600} \text{ h} \times 0.233 \text{ kg/kWh} \approx 4.6 \times 10^{-6} \text{ kg} \quad (10)$$

Total CO₂ per method is the sum of per-evaluation emissions:

$$\text{CO}_{2,\text{total}} = \sum_{i=1}^N \text{CO}_{2,i}$$

(11)

B. Method

Hyperparameter Search Space. The search space is defined as follows:

- **Number of layers:** {1, 2, 3, 4, 5, 6}
- **Neurons per layer:** {32, 64, 128, 256, 512}
- **Learning rate:** { 10^{-4} , 5×10^{-4} , 10^{-3} , 5×10^{-3} , 10^{-2} , 5×10^{-2} }
- **Batch size:** {16, 32, 64, 128, 256}

This yields $6 \times 5 \times 6 \times 5 = 900$ possible combinations, shared identically across all three methods for fair comparison.

Chromosome Encoding. Each candidate solution is represented by a chromosome consisting of four integer decision variables:

$$\mathbf{x} = [x_0, x_1, x_2, x_3] \quad (12)$$

where $x_0 \in \{0, \dots, 5\}$ encodes the number of hidden layers, $x_1 \in \{0, \dots, 4\}$ encodes the number of neurons per layer, $x_2 \in \{0, \dots, 5\}$ encodes the learning rate, and $x_3 \in \{0, \dots, 4\}$ encodes the batch size.

Integer encoding is employed because pymoo's NSGA-II implementation operates directly on integer-valued decision variables. During evaluation, each index is mapped to its corresponding hyperparameter value. For example, the chromosome

[2, 1, 3, 0]

corresponds to a network with three hidden layers, 64 neurons per layer, learning rate 5×10^{-3} , and batch size 16.

pymoo Framework. The NSGA-II algorithm is implemented using pymoo [11], a Python library specifically designed for multi-objective optimization. pymoo provides a modular architecture in which the optimization problem and the algorithm are defined separately and combined through a single `minimize()` call:

```
1 # Define the problem
2 problem = HyperparameterOptimizationProblem()
3
4 # Configure the algorithm
5 algorithm = NSGA2(
6     pop_size=50,
7     sampling=IntegerRandomSampling(),
8     crossover=TwoPointCrossover(prob=0.8),
9     mutation=PM(prob=0.1, eta=3, vtype=int),
10    eliminate_duplicates=True
11 )
12
13 # Run the optimization
14 result = minimize(problem, algorithm,
15                  termination, seed=42)
```

Listing 1: pymoo problem and algorithm setup

The optimization problem is formulated as a custom subclass of pymoo's `Problem` base class, named `HyperparameterOptimizationProblem`. This class specifies four decision variables ($n_{var} = 4$), three objectives ($n_{obj} = 3$), no constraints ($n_{constr} = 0$), and integer bounds corresponding to the hyperparameter index ranges ($x_l = [0, 0, 0, 0]$, $x_u = [5, 4, 5, 4]$). The `_evaluate()` method is overridden to implement the actual fitness evaluation: for each chromosome in the population, it decodes the integer indices to hyperparameter values, trains an MLP on the training set, evaluates it on the validation set, measures training time and CO₂ emissions using CodeCarbon, and returns the three objective values through pymoo's `out['F']` interface — a dictionary entry that pymoo reads back to perform non-dominated sorting and selection.

The `minimize()` function orchestrates the entire evolutionary process — initializing the population, calling `_evaluate()` each generation on newly created offspring, applying genetic operators, performing non-dominated sorting and crowding distance computation, and selecting survivors. Upon termination after 30 generations, the algorithm returns the final Pareto front in `result.F`, containing one row of objective values (ϵ, t, c) for each non-dominated solution, and `result.X`, containing the corresponding chromosome encodings. The number of Pareto-optimal solutions is not fixed and depends on the evolutionary outcome of a particular run.

Algorithm 1 NSGA-II for Carbon-Aware HPO

Require: $N = 50$, $G = 30$, $p_c = 0.8$, $p_m = 0.1$, $\eta = 3$

Ensure: Pareto-optimal hyperparameter configurations

Initialize P_0 with N random integer chromosomes

Evaluate P_0 : compute (ϵ, t, c) on validation set

for $g = 1$ **to** G **do**

 Non-dominated sorting $\rightarrow$ rank fronts

 Compute crowding distance within each front

 Select parents via rank + crowding distance

 Apply Two-Point Crossover ($p_c = 0.8$)

 Apply Polynomial Mutation ($p_m = 0.1$, $\eta = 3$)

 Evaluate offspring Q_g : compute (ϵ, t, c)

 Eliminate duplicate chromosomes

 Combine $P_g \cup Q_g$, keep best $N \rightarrow P_{g+1}$

end for

return Pareto front of P_G

Genetic Operators. Discrete-compatible operators are used since all genes are categorical integers:

- **Sampling — IntegerRandomSampling:** The initial population of $N = 50$ chromosomes is generated by uniformly sampling random integers within bounds $[x_l, x_u]$ for each gene, ensuring maximum diversity at the start of evolution.
- **Crossover — TwoPointCrossover** ($p_c = 0.8$): Two cut points are selected randomly along the chromosome and the segment between them is exchanged between two parent chromosomes, producing two offspring. With probability $p_c = 0.8$, crossover is applied — otherwise offspring are direct copies of their parents, preserving well-performing configurations. TwoPointCrossover is chosen over Simulated Binary Crossover (SBX) because SBX is designed for continuous variables and produces fractional offspring values that require integer casting, causing information loss. TwoPointCrossover operates directly in the discrete integer space, always producing valid integer indices within the defined bounds — ensuring every offspring maps to a legitimate hyperparameter configuration without any post-processing. With only four genes per chromosome, there are three possible cut positions between adjacent genes, yielding $\binom{3}{2} = 3$ distinct cut-point pairs and therefore three distinct swappable segments: a single leading gene, a middle pair, or a single trailing gene. Crucially, the ability to isolate and swap a *middle* segment — inheriting, for example, the neurons and learning rate genes from one parent while keeping the layers and batch size genes from the other — is a recombination pattern that one-point crossover structurally cannot produce, making two-point crossover strictly more expressive for this chromosome representation.

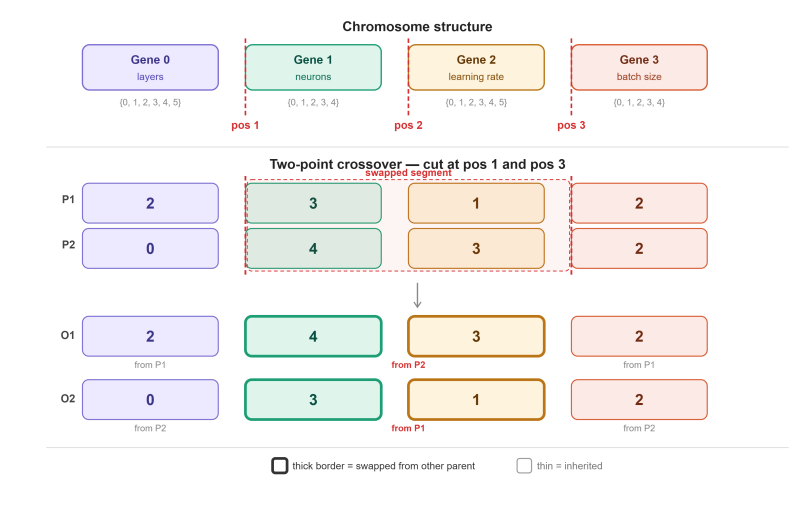


Figure 1. Two-Point Crossover applied to a 4-gene integer chromosome encoding MLP hyperparameters. *Top*: chromosome structure showing the four genes and their index ranges, with three possible cut positions between adjacent genes. *Bottom*: example crossover with cut points at pos 1 and pos 3 — the middle segment (genes 1 and 2) is exchanged between the two parents, producing two offspring. Thick borders indicate the swapped segment; thin borders indicate genes inherited unchanged.

- **Mutation — Polynomial Mutation** ($p_m = 0.1$, $\eta = 3$): Each of the four genes in a chromosome is independently perturbed with probability $p_m = 0.1$, meaning on average 0.4 genes are mutated per offspring. The distribution index η controls the spread of perturbation — lower values produce larger jumps, higher values produce smaller ones.

The default value of $\eta = 20$ is designed for large continuous spaces and produces perturbations smaller than one integer unit, which after casting to integer produces no change at all — making mutation completely ineffective in our discrete search space. We therefore set $\eta = 3$ to guarantee that mutation produces real index changes.

It is worth noting that in our integer-encoded search space, any index change is semantically meaningful regardless of its size. For example, a one-step jump in the learning rate gene from index 1 to index 2 corresponds to a change from 5×10^{-4} to 10^{-3} — a twofold increase that significantly affects training dynamics. Similarly, a one-step jump in the neurons gene from index 1 to index 2 doubles the network width from 64 to 128 neurons. Therefore, the primary role of $\eta = 3$ is not to control the semantic magnitude of hyperparameter changes — which are always meaningful — but simply to ensure that mutation produces integer-level perturbations at all, allowing NSGA-II to explore new configurations and maintain population diversity throughout evolution.

- **Duplicate Elimination:**

pymoo's `eliminate_duplicates=True` option prevents identical chromosomes from appearing simultaneously within the same population whenever possible. However, previously evaluated configurations may still reappear in later generations

due to selection, crossover, and mutation. Therefore duplicate elimination alone does not prevent redundant evaluations across the entire optimization run.

Non-Dominated Sorting. Solution **A** dominates **B** when:

$$\forall m : f_m(\mathbf{A}) \leq f_m(\mathbf{B}) \quad \text{and} \quad \exists m : f_m(\mathbf{A}) < f_m(\mathbf{B}) \quad (13)$$

Solutions dominated by no other form the Pareto front (Rank 1). Solutions dominated only by Rank 1 form Rank 2, and so on.

Crowding Distance. Within each rank, the crowding distance of solution i is computed as the sum of normalized distances to its nearest neighbors across all objectives:

$$d_i = \sum_{m=1}^M \frac{f_m^{i+1} - f_m^{i-1}}{f_m^{\max} - f_m^{\min}} \quad (14)$$

where f_m^{i+1} and f_m^{i-1} are the objective values of the neighboring solutions on either side of solution i along objective m , and $f_m^{\max} - f_m^{\min}$ is the objective range used for normalization. Solutions at the boundary of each rank are assigned $d_i = \infty$, ensuring they are always retained.

Crowding distance serves two roles in NSGA-II:

- **Parent selection:** when two solutions share the same rank, the one with higher crowding distance is preferred — favoring isolated solutions over clustered ones, promoting diverse offspring.
- **Survivor selection:** when the last rank overflows the population budget, solutions with the highest crowding distance are retained and the most clustered ones are discarded, preserving spread across the Pareto front.

This mechanism is responsible for NSGA-II's ability to maintain diversity along the Pareto front by preserving solutions in sparsely populated regions of the objective space.

Evaluation Cache.

To eliminate redundant MLP training when NSGA-II revisits previously evaluated configurations, an evaluation cache is implemented as a Python dictionary mapping chromosome tuples to their objective values. When a chromosome is evaluated for the first time, the MLP is trained and the result is stored in the cache. If the same configuration appears again in a later generation, the stored objective values are returned immediately without retraining the model:

```
1 evaluation_cache = {}
2
3 def evaluate(chromosome):
4     key = tuple(int(v) for v in chromosome)
5
6     if key in evaluation_cache:
7         return evaluation_cache[key]
8
9     result = train_mlp(chromosome)
10    evaluation_cache[key] = result
11
12    return result
```

Listing 2: Evaluation cache implementation

The cache does not alter NSGA-II's evolutionary behaviour. Selection, crossover, mutation, non-dominated sorting, and crowding distance remain unchanged; only the computational cost of repeated evaluations is removed.

It is important to distinguish duplicate elimination from evaluation caching. Duplicate elimination reduces repeated chromosomes within a generation, whereas the cache prevents re-training whenever a previously evaluated configuration is encountered at any point during the search.

Across all runs, NSGA-II generated approximately 1,500 evaluation requests (50 initial solutions plus 50×30 offspring). However, only about 450 unique hyperparameter configurations were explored. The cache therefore eliminated roughly 70% of potential MLP trainings while preserving identical objective values and Pareto-front quality.

Hypervolume Indicator. The quality of each algorithm's Pareto front is measured using the Hypervolume (HV) indicator [12]. HV measures the volume of objective space dominated by the Pareto front relative to a reference point — higher values indicate better overall front quality, capturing both convergence toward good solutions and diversity across the objective space. To ensure fair comparison across all three methods, the following procedure is applied:

$$HV(S, r) = \lambda \left(\bigcup_{s \in S} \prod_{i=1}^m [s_i, r_i] \right) \quad (15)$$

where λ denotes the Lebesgue measure, m is the number of objectives, and r is a reference point that is dominated by all solutions in S . A larger hypervolume indicates a better approximation of the true Pareto front in terms of both convergence and diversity. In this work, objectives were normalized to $[0, 1]$ prior to computation and the reference point was set to $r = (1.1, 1.1, 1.1)$, placing it slightly beyond the worst observed normalized values

- 1) **Normalize** all Pareto fronts to $[0, 1]$ using min-max scaling computed globally over all evaluated configurations across all three methods:

$$\hat{f} = \frac{f - f_{\min}}{f_{\max} - f_{\min}} \quad (16)$$

Using global ranges ensures that all methods are compared on the same scale, preventing any method from artificially inflating its hypervolume through per-method normalization.

- 2) **Clip** normalized values strictly to $[0, 1]$ to guarantee validity. Since training time and CO₂ emissions depend on CPU load at evaluation time, a Pareto solution may occasionally produce a value slightly outside the global normalization range, yielding a normalized value marginally above 1 or below 0. Clipping prevents this from causing incorrect hypervolume computation.
- 3) **Compute** hypervolume using the fixed reference point $r = [1.1, 1.1, 1.1]$, placed 10% beyond the worst normalized objective value in each dimension, ensuring the reference point strictly dominates all Pareto solutions. This 10% margin is a standard choice in the multi-objective optimization literature [13]. The maximum possible hypervolume is $1.1^3 = 1.331$, corresponding to a hypothetical single solution at the origin $[0, 0, 0]$ that dominates the entire normalized objective space.

Selection of the Final Pareto Solution. The Pareto front contains multiple non-dominated solutions, each representing a different trade-off between validation error, training time, and CO₂ emissions. To select a single configuration for final evaluation, a weighted fitness function is applied to the normalized objectives:

$$f(\mathbf{x}) = w_1 \cdot \hat{e} + w_2 \cdot \hat{t} + w_3 \cdot \hat{c} \quad (17)$$

where $\hat{e}$, $\hat{t}$, and $\hat{c}$ denote the normalized validation error, training time, and CO₂ emissions, respectively. Equal weights $w_1 = w_2 = w_3 = \frac{1}{3}$ are used, assigning the same importance to all three objectives. The weights satisfy $w_1 + w_2 + w_3 = 1$, ensuring that the fitness score remains within $[0, 1]$. Lower fitness values indicate better overall trade-offs.

The fitness function is applied only after the search process has completed to rank Pareto-optimal solutions and select a final configuration. During optimization, NSGA-II treats the three objectives independently and does not use the weighted fitness function.

Baseline Methods. Two baseline methods are implemented for comparison, both using the identical search space (900 combinations), training set, validation set, and evaluation conditions as NSGA-II.

Grid Search. Grid Search performs an exhaustive evaluation of all possible combinations within a predefined hyperparameter grid, as illustrated in Figure 2. While this guarantees finding the optimal configuration within the search space, it suffers from the curse of dimensionality — the number of evaluations grows exponentially with the number of hyperparameters, making it computationally infeasible for large search spaces. In our work, Grid Search evaluates all 900 combinations, recording training time and CO₂ emissions per configuration.

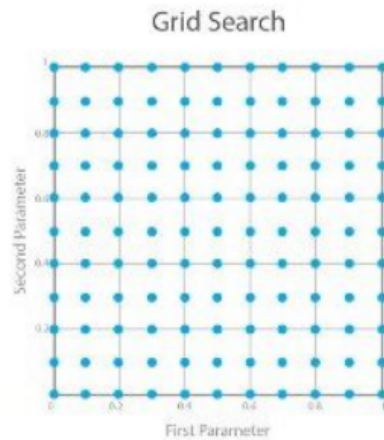


Figure 2. Grid Search exhaustively evaluates all combinations in the hyperparameter space, covering every possible configuration.

Random Search. Rather than evaluating all combinations exhaustively, Random Search samples configurations uniformly at random within the same search space, as shown in Figure 3. This approach is substantially more efficient and generalizes naturally to continuous and mixed hyperparameter spaces. Studies have shown that Random Search can match or outperform Grid Search when only a subset of hyperparameters significantly affects model performance [4]. In our work, Random Search evaluates 100 randomly sampled configurations — just 11% of the full search space — achieving competitive results at a fraction of the computational cost.

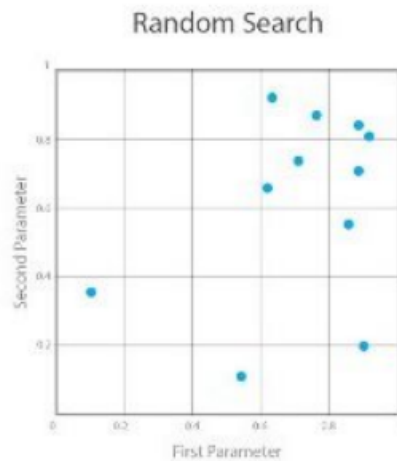


Figure 3. Random Search samples configurations uniformly at random within the hyperparameter space, covering a representative subset of possible combinations.

IV. RESULTS

A. Experimental Setup

The Wine Quality dataset is loaded from the UCI Machine Learning Repository using the `ucimlrepo` Python library and partitioned into three disjoint sets using stratified sampling: 70% training (4,550 samples), 15% validation (972 samples), and 15% test (975 samples). The validation set is used exclusively for objective evaluation during hyperparameter search, while the test set is reserved for final model evaluation and is used only once after the best configuration has been selected.

The NSGA-II algorithm is configured with the following parameters, as specified in the project requirements: population size $N = 50$, number of generations $G = 30$, crossover probability $p_c = 0.8$, mutation probability $p_m = 0.1$, and distribution index

$\eta = 3$ for Polynomial Mutation. Three objectives are optimized simultaneously: validation error ϵ , training time t , and CO₂ emissions c .

The MLP is trained with `max_iter=30` and `early_stopping=False` during the search process, ensuring that all 4,550 training samples are used consistently in every evaluation. An evaluation cache is employed to avoid redundant model training by reusing previously computed objective values for duplicate hyperparameter configurations. Across five independent NSGA-II runs, the cache reduced the average number of actual MLP trainings from 1,499 evaluation requests to approximately 437 unique model trainings.

To account for stochastic variability in evolutionary search, all experiments are repeated five independent times and average results are reported unless otherwise stated. After the final configuration is selected, the model is retrained with `max_iter=100` on the combined training and validation sets (5,522 samples) to maximize test accuracy.

To select a single configuration from each Pareto front for final evaluation, a weighted fitness function is applied to the normalized objectives. Equal weights $w_1 = w_2 = w_3 = \frac{1}{3}$ are used, reflecting the core motivation of this work: predictive accuracy, computational efficiency, and carbon emissions are treated as equally important objectives. Equal weighting prevents any single objective from dominating the selection process and favours balanced trade-offs between accuracy, speed, and environmental impact.

For multi-objective comparison, all Pareto fronts are normalized to the interval $[0, 1]$ using global min–max scaling computed over all evaluated configurations from all methods. Hypervolume is then computed using the reference point

$$\mathbf{r} = [1.1, 1.1, 1.1]$$

which lies 10% beyond the worst normalized objective value in each dimension, ensuring that all Pareto fronts are fully enclosed by the reference region used for hypervolume calculation.

B. Pareto Front Analysis

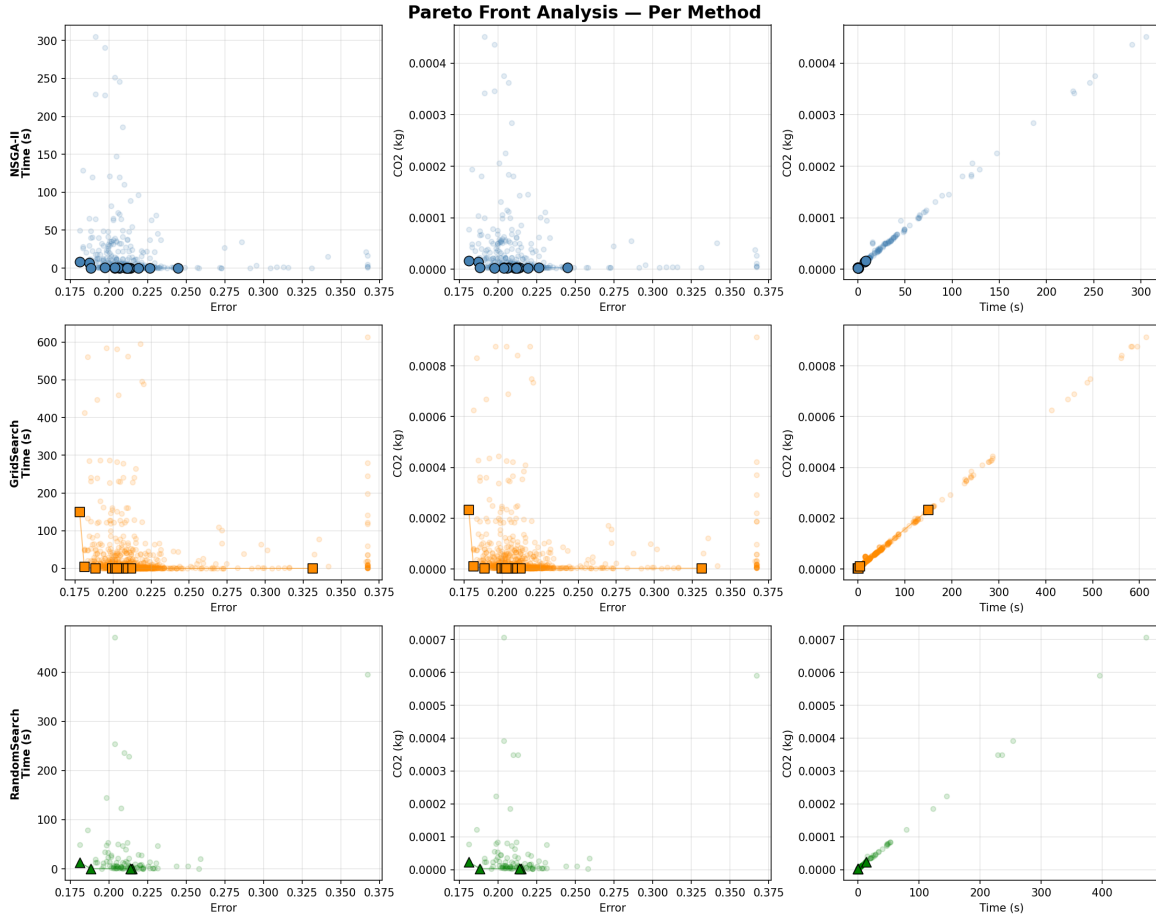


Figure 4. Representative Pareto fronts for NSGA-II, Grid Search, and Random Search. Large markers denote Pareto-optimal solutions, while transparent markers show all evaluated configurations. The Time–CO₂ projection highlights the strong correlation between training time and carbon emissions.

Figure 4 presents representative Pareto fronts from a single experimental run. Minor variations between runs are expected due to fluctuations in training time and the resulting CO₂ estimates, whereas the quantitative metrics reported in the tables correspond to averages across multiple runs.

Time vs CO₂ correlation. The right column shows a near-perfect linear relationship between training time and CO₂ emissions across all methods. This is expected because CodeCarbon estimates emissions from hardware power consumption and wall-clock execution time, making longer training runs produce higher carbon emissions.

NSGA-II exploration. The NSGA-II row (top) shows a diverse set of Pareto-optimal solutions spanning a wide range of validation errors and computational costs. Despite evaluating only a subset of the 900 possible hyperparameter configurations, NSGA-II identifies trade-off solutions that closely resemble those found by exhaustive Grid Search. This demonstrates that evolutionary search can efficiently approximate the reference Pareto front while requiring substantially fewer unique evaluations.

Grid Search coverage. The Grid Search row (middle) exhibits the widest coverage of the objective space, consistent with its exhaustive evaluation of all 900 hyperparameter combinations. Most Pareto-optimal solutions correspond to relatively small MLP architectures with low training time and low CO₂ emissions. For example, the architecture with two hidden layers of 32 neurons achieves a validation accuracy of 81.17% in only 0.19 s, whereas increasing the architecture to two hidden layers of 256 neurons improves validation accuracy to only 81.89% while increasing training time to 4.62 s (approximately 25× longer). Expanding further to six hidden layers of 512 neurons increases validation accuracy only slightly to 82.20%, but requires 149.90 s of training (over 30× longer than the 256-neuron model and approximately 800× longer than the 32-neuron model). Consequently, only a small number of these computationally expensive architectures remain non-dominated because their lower validation error compensates for their much higher computational cost, while many intermediate configurations are dominated.

Random Search exploration. The Random Search row (bottom) covers a more limited portion of the objective space than either NSGA-II or Grid Search, reflecting its budget of only 100 randomly sampled configurations. Consequently, its Pareto front contains fewer non-dominated solutions, providing fewer alternative trade-off configurations for balancing predictive performance, training time, and CO₂ emissions.

C. Pareto Front Quality and Computational Cost

Table III: Average Pareto front quality and computational cost across five runs (mean $\pm$ standard deviation). Bold values indicate the best result per metric.

Method	HV $\uparrow$	Time (min) $\downarrow$	CO ₂ (kg) $\downarrow$	Evaluations	Front Size $\uparrow$
NSGA-II	1.310 $\pm$ 0.001	115.5 $\pm$ 19.3	0.0092 $\pm$ 0.0016	437.4 $\pm$ 22.2	11.8 $\pm$ 4.5
Grid Search	1.323 $\pm$ 0.004	444.0 $\pm$ 55.7	0.0441 $\pm$ 0.0176	900	11.2 $\pm$ 2.8
Random Search	1.309 $\pm$ 0.001	50.1 $\pm$ 8.0	0.0043 $\pm$ 0.0006	100	3.6 $\pm$ 0.9

Table III summarises the Pareto-front quality and computational cost of each method averaged across five independent runs. Grid Search achieves the highest average hypervolume (1.323 $\pm$ 0.004), as its exhaustive evaluation of all 900 configurations recovers the true Pareto front within the predefined search space. However, the difference in hypervolume between Grid Search and NSGA-II is small (approximately 1%), indicating that NSGA-II closely approximates the reference front while requiring roughly half the evaluations, substantially lower runtime (115.5 vs. 444.0 minutes), and significantly lower CO₂ emissions (0.0092 vs. 0.0441 kg).

Random Search achieves a hypervolume very close to that of NSGA-II (1.309 vs. 1.310) while requiring the fewest evaluations and the lowest runtime and CO₂ emissions. However, its limited evaluation budget of 100 configurations produces an average Pareto front of only 3.6 $\pm$ 0.9 non-dominated solutions, compared with 11.8 $\pm$ 4.5 for NSGA-II and 11.2 $\pm$ 2.8 for Grid Search. This illustrates a limitation of hypervolume when interpreted in isolation: despite covering a similar volume of the objective space, the Random Search front is substantially smaller and therefore provides fewer trade-off options for practitioners. This occurs because a small number of well-spread solutions can dominate a large volume of objective space even without dense coverage — particularly in this setting where training time and CO₂ emissions are highly correlated, effectively reducing the three-dimensional objective space to two independent dimensions and making it easier for randomly sampled configurations to achieve good spread.

Overall, NSGA-II offers the best balance between Pareto-front quality, Pareto-front size, and computational efficiency. Despite achieving a hypervolume only approximately 1% below Grid Search, it produces a similarly rich Pareto front while requiring substantially fewer evaluations, lower runtime, and lower CO₂ emissions. Random Search, although computationally efficient, generates much smaller Pareto fronts and therefore offers fewer trade-off solutions for decision makers.

D. Hypervolume Interpretation

Figure 5 illustrates the normalized Pareto fronts used for hypervolume computation across the three methods. Although Grid Search, NSGA-II, and Random Search produce markedly different numbers of Pareto-optimal solutions—12, 16, and 4, respectively, in this representative run—their hypervolume values remain remarkably similar.

This behaviour can be explained by the geometry of the objective space. Most Pareto-optimal solutions are concentrated near the lower bounds of the training time and CO₂ objectives, where computationally efficient MLP architectures achieve competitive validation errors. Furthermore, as shown in the rightmost column of Figure 5, training time and CO₂ emissions exhibit a nearly perfect linear relationship. Consequently, the three-objective problem effectively behaves as approximately two independent objectives under the experimental conditions considered in this study.

As a result, even a relatively small number of well-positioned Pareto-optimal solutions can dominate nearly the same region of the objective space as a much denser Pareto front. Random Search therefore achieves a hypervolume comparable to NSGA-II despite producing substantially fewer non-dominated solutions, because its Pareto front spans a similar boundary of the objective space.

However, hypervolume measures only the dominated volume and does not account for the density of solutions along the Pareto front. While Random Search achieves a comparable hypervolume, its sparse front provides considerably fewer alternative trade-offs between validation error, training time, and CO₂ emissions than NSGA-II or Grid Search. These observations demonstrate that, for this problem, hypervolume should be interpreted together with Pareto front size to provide a more complete assessment of multi-objective search quality.

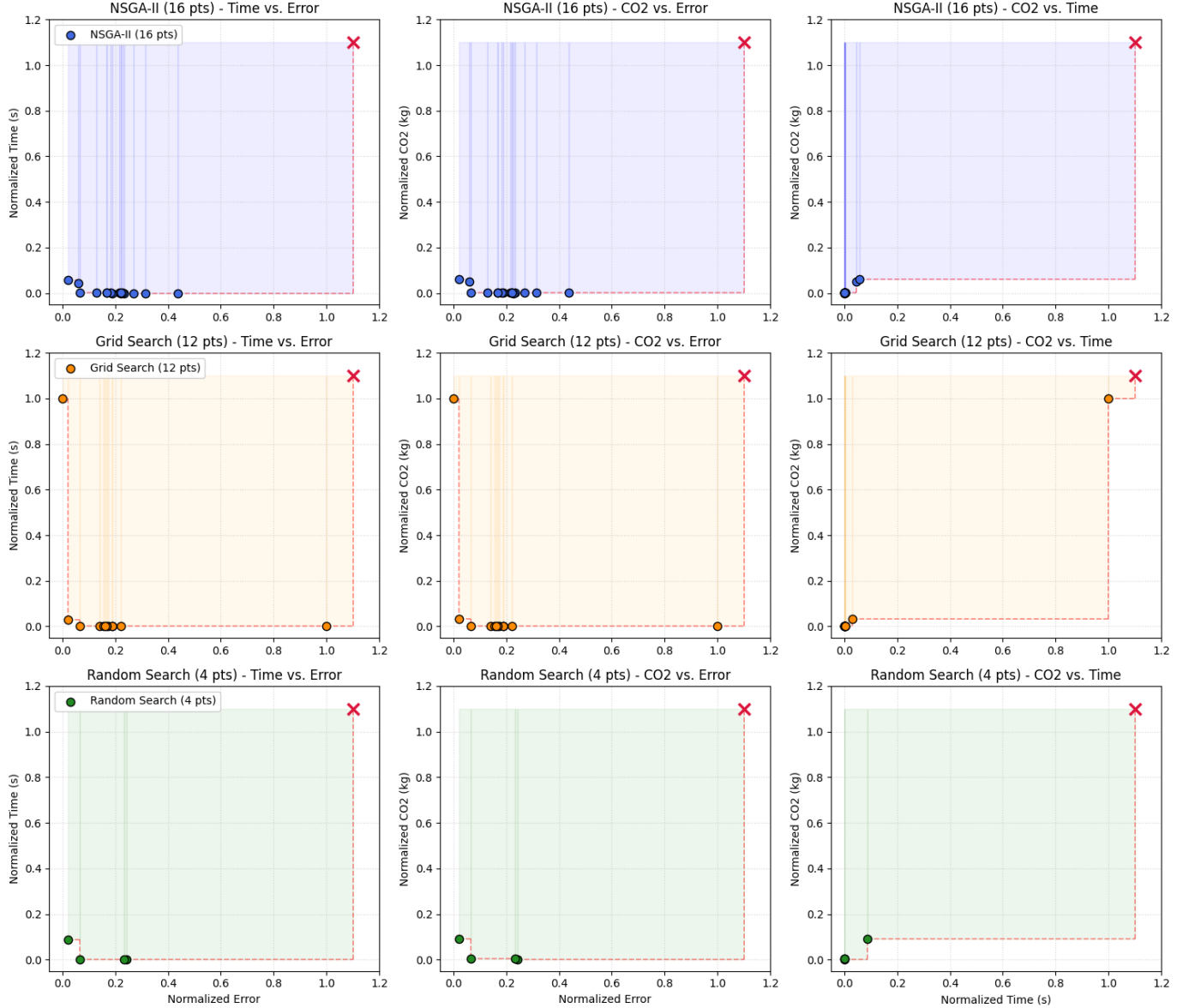


Figure 5. Normalized Pareto fronts used for hypervolume computation. The shaded region represents the dominated hypervolume with respect to the reference point $\mathbf{r} = [1.1, 1.1, 1.1]$. Although Random Search produces far fewer Pareto-optimal solutions than NSGA-II and Grid Search, it dominates a similar region of the objective space, resulting in a comparable hypervolume.

E. Selection of the Final Configuration

A weighted fitness function with equal weights $w_1 = w_2 = w_3 = \frac{1}{3}$ is applied to the normalized Pareto front of each method to select a single configuration for final evaluation. For each of the five independent runs, the solution with the lowest weighted fitness is selected from the Pareto front. Table IV reports the best fitness value and corresponding test accuracy obtained across the five runs for each method.

Table IV: Best selected configuration across five runs. Fitness is computed using equal weights $w_1 = w_2 = w_3 = \frac{1}{3}$ over normalized objectives. Lower fitness and higher test accuracy are better.

Method	Best Fitness ↓	Test Accuracy (%) ↑
NSGA-II	0.0067	75.69
Grid Search	0.0062	79.69
Random Search	0.0074	80.92

Fitness Results. Grid Search achieves the lowest fitness value (0.0062), while NSGA-II achieves a very similar value (0.0067). Since the fitness function equally weights validation error, training time, and CO₂ emissions, these results indicate that both methods identify configurations with a similar overall balance between predictive performance, computational efficiency, and environmental impact. Random Search obtains the highest fitness value (0.0074), reflecting a less favourable overall trade-off among the three objectives.

Test Accuracy Results. The highest observed test accuracy is obtained by Random Search (80.92%), followed by Grid Search (79.69%) and NSGA-II (75.69%). This does not contradict the fitness results because the two metrics measure different quantities. The fitness function is computed during the search using validation error, training time, and CO₂ emissions, whereas test accuracy is measured only after the selected configuration has been retrained on the combined training and validation sets. As a result, the configuration with the highest test accuracy is not necessarily the one with the lowest fitness value.

V. CONCLUSION

This work proposed a carbon-aware hyperparameter optimization framework based on the Non-dominated Sorting Genetic Algorithm II (NSGA-II), which simultaneously optimizes validation error, training time, and CO₂ emissions. By treating predictive performance, computational efficiency, and environmental impact as equally important objectives, the framework identifies balanced trade-off solutions rather than optimizing a single objective in isolation.

Experiments on the Wine Quality dataset demonstrated that NSGA-II achieves near-optimal Pareto-front quality while requiring substantially fewer evaluations than exhaustive Grid Search. Across five independent runs, NSGA-II achieved an average hypervolume of 1.310 ± 0.001 , only approximately 1% below Grid Search (1.323 ± 0.004), despite evaluating only 437.4 ± 22.2 unique configurations compared with Grid Search's 900. Furthermore, NSGA-II produced an average Pareto front of 11.8 ± 4.5 non-dominated solutions, closely matching Grid Search (11.2 ± 2.8), demonstrating that evolutionary search can recover a rich set of trade-off solutions using roughly half the evaluation budget.

Random Search achieved the lowest runtime (50.1 ± 8.0 minutes) and CO₂ emissions (0.0043 ± 0.0006 kg), as well as the highest observed test accuracy (80.92%). However, its limited budget of 100 evaluations produced an average Pareto front of only 3.6 ± 0.9 non-dominated solutions. Although its hypervolume was comparable to that of NSGA-II, the much smaller Pareto front provides fewer alternative trade-off solutions, showing that hypervolume alone does not fully characterize multi-objective search performance.

An additional contribution of this work is the evaluation cache, which reduced the average number of actual MLP trainings from 1,499 evaluation requests to approximately 437 unique model trainings by reusing previously computed objective values for duplicate configurations. This substantially reduced runtime and CO₂ emissions without altering the optimization behaviour of the algorithm, and represents a practical engineering contribution applicable to evolutionary hyperparameter optimization.

A notable observation is the close hypervolume values obtained by all three methods (1.323, 1.310, and 1.309 for Grid Search, NSGA-II, and Random Search, respectively). As shown in the Pareto front visualizations, training time and CO₂ emissions exhibit a near-perfect linear relationship, effectively reducing the optimization problem to two independent objectives under the experimental conditions. Furthermore, the best solutions are concentrated near the lower bounds of the training time and CO₂ objectives, allowing all three methods to dominate similar regions of the objective space. As a result, hypervolume alone does not distinguish between NSGA-II's average Pareto front of 11.8 ± 4.5 solutions and Random Search's much smaller front of 3.6 ± 0.9 solutions. This highlights the importance of considering Pareto-front size alongside hypervolume when evaluating multi-objective optimization methods.

Overall, the results demonstrate that NSGA-II provides the best balance between Pareto-front quality, Pareto-front size, and evaluation efficiency among the methods considered. By recovering a near-optimal Pareto front using approximately half the unique model evaluations required by exhaustive Grid Search, the proposed framework demonstrates the effectiveness of evolutionary multi-objective optimization for carbon-aware hyperparameter tuning. As hyperparameter search spaces continue to grow and environmental considerations become increasingly important, carbon-aware multi-objective optimization represents a promising direction for more sustainable machine learning.

REFERENCES

- [1] V. Valtchanov, "Hyperparameter optimization in machine learning," *arXiv preprint*, 2021.
- [2] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019, pp. 3645–3650.
- [3] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling wine preferences by data mining from physicochemical properties," *Decision Support Systems*, vol. 47, no. 4, pp. 547–553, 2009.
- [4] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, pp. 281–305, 2012.
- [5] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian optimization of machine learning algorithms," in *Advances in Neural Information Processing Systems*, vol. 25, 2012.

- [6] Y. Ozaki, Y. Tanigaki, S. Watanabe, and M. Onishi, "Multiobjective tree-structured parzen estimator for computationally expensive optimization problems," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2020, pp. 533–541.
- [7] J. H. Holland, *Adaptation in Natural and Artificial Systems*. MIT Press, 1992.
- [8] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.
- [9] Z. Lu, I. Whalen, V. Boddeti, Y. Dhebar, K. Deb, E. Goodman, and W. Banzhaf, "NSGA-Net: Neural architecture search using multi-objective genetic algorithm," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2019, pp. 419–427.
- [10] V. Courty, K. Lottick, Goyal-Kamal, P. Nanayakkara, S. Luccioni, P. Goyal, L. Tazi, A. Kotto, R. Sherali, T. Toma, R. Sherali, M. Moran, Y. Boquet, and K. Lottick, "CodeCarbon: Estimate and track carbon emissions from machine learning computing," *Journal of Open Source Software*, vol. 8, no. 85, p. 5060, 2023.
- [11] J. Blank and K. Deb, "pymoo: Multi-objective optimization in python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.
- [12] E. Zitzler and L. Thiele, "Multiobjective optimization using evolutionary algorithms — a comparative case study," *Parallel Problem Solving from Nature*, pp. 292–301, 1998.
- [13] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, and V. G. da Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," in *IEEE Transactions on Evolutionary Computation*, vol. 7, no. 2, 2003, pp. 117–132.